2. Build

All building goes through `v001/build.sh`; TreeSyncBuild, `verify.sh` and `bootstrap.sh` all call it. Run everything from `jac317/v001`.

TreeSync is four programs, all in the **TreeSyncApp** folder of your app list:

Program	Job	Lives in	Runs as
TreeSyncBuild	build and test, with bootstrap's requirements check (2.3)	the source tree	you
TreeSyncInstall	install the release build for root, or remove it (2.6)	the source tree	you (asks for the password to install)
TreeSyncRoot-GCC, **TreeSyncRoot-Clang**	sync as root, any folder (3.9)	`/usr/local/bin`	you; each sync as root
TreeSyncHome-GCC, **TreeSyncHome-Clang**	sync as you, inside your home (3.1)	`/usr/local/bin`	you

TreeSync is built with both GCC and Clang, and both builds can be installed side by side: each sync program comes once per compiler, and you install one or both.

2.1 What gets built

Program	What it is
`treesync-root`	**TreeSyncRoot**: the sync window, every sync as root
`treesync-home`	**TreeSyncHome**: the same window, every sync as you, inside your home
`treesync-cli`	The command-line program; does every sync (both GUIs start it)
`treesync-build`	**TreeSyncBuild**: builds and tests TreeSync (2.3); never installed itself
`treesync-install`	**TreeSyncInstall**: installs the release build for root (2.6); never installed itself
`treesync-test`	The core tests (built with `--test`)

Each compiler has its own folder: `out/gcc/` and `out/clang/`. The two builds are the same program; TreeSync is also a proving ground for GCC 15 and Clang 21 with C++26.

2.2 build.sh

```
./build.sh --compiler gcc                # debug build with gcc 15
./build.sh --compiler clang              # debug build with clang 21 (the default compiler)
./build.sh --compiler gcc --release      # optimized build: use this for real syncs
./build.sh --compiler gcc --test         # build, then run the core tests
./build.sh --compiler gcc --release --test
./build.sh --compiler gcc --clean        # compile everything, not only what changed
```

Option	What it does
`--compiler gcc` / `clang`	Which compiler (default clang)
`--release`	Optimized build (default: debug)
`--test`	Also build `treesync-test` and run it; the exit code is non-zero if a test fails
`--clean`	Rebuild everything (fair build times when comparing compilers)
`--tsan`	Instead of the build: the tests under ThreadSanitizer (clang), into `out/clang-tsan/`
`--asan`	The tests under AddressSanitizer + UBSan, then `treesync-cli --bench` under them
`--valgrind`	The tests under valgrind memcheck (slow)
`--helgrind`	The tests under valgrind helgrind (very slow)

Debug or release? Debug builds check more and are easier to debug, but are much slower: on many small files about 4 times the CPU. Always run real syncs with a **release** build. `build.sh` remembers which type is in each `out/` folder and links the programs again when you switch, so a debug build never leaves a release program behind or the other way round.

A test run ends with a line such as `passed: 290/290 checks`. A debug build skips one test (a 4.5 GiB sparse copy, too slow without optimization) and says so; a release test run has all of them (`passed: 303/303 checks`).

2.3 TreeSyncBuild

Building and testing have a program of their own, separate from the ones that sync. Start it as yourself: **TreeSyncBuild** in the TreeSyncApp folder of your app list (`bootstrap.sh` puts it there), or `v001/out/gcc/treesync-build`. It lives in the source tree and is never installed; as root it refuses to start (root runs TreeSync, it does not build it). It also does bootstrap's part on a machine that has been set up: **This machine** runs `bootstrap.sh --check`, and **Open terminal to fix...** opens `bootstrap.sh` in a terminal.

- **Compiler**: gcc, clang, or both one after the other.
- **Type**: Debug or Release.

- **Run the tests (treesync-test) after building**: the one switch for testing. Unticked, **Build** only builds. Ticked, the tests run, plus the checked runs below it:

    - Also under ThreadSanitizer (clang; data races)
    - Also under AddressSanitizer + UBSan (memory errors, undefined behaviour)
    - Also under valgrind memcheck (slow)
    - Also under valgrind helgrind (very slow)

  These keep their ticks while "Run the tests" is off; they are greyed out and skipped. - **Clean build**: compile everything. - **Compare the compilers**: versions, C++26 features, `import std`, speed and size, side by side. - **Open TreeSyncInstall after a good release build**: when every step went well, TreeSyncInstall (2.6) opens, ready to install what was just built. Only for a release build (a debug build is not installed). - **Display**: Wayland (recommended) or X11, for all four programs. - **This machine**: whether the requirements are present (**Check again**, **Open terminal to fix...**).

The command lines it will run are shown at the bottom. The buttons:

- **Build**: runs them one after another and shows each one's output; the results screen sums them up (kept in `Build-Results.txt` in the logs folder).
- **Install...**: opens TreeSyncInstall (2.6), for the builds already in `out/`.
- **Run TreeSyncHome-GCC (just built) / Run TreeSyncHome-Clang (just built)**: starts the TreeSyncHome this window just built (from `v001/out/gcc` or `out/clang`, not the installed one), as you, in a window of its own, to try it before installing; TreeSyncBuild stays open.
- **Help** (F1): this chapter.

Your choices are saved in `~/.config/treesync/build`.

2.4 The sanitizer and valgrind runs

These find bugs the normal tests cannot see: data races between the copy threads, memory errors, undefined behaviour. They take much longer than the normal tests, so run them when threading or memory handling changed:

```
./build.sh --compiler clang --tsan
./build.sh --compiler gcc --asan
./build.sh --compiler gcc --valgrind
./build.sh --compiler gcc --helgrind
```

They need the optional `checks` requirement (see 1.4). Each builds into its own folder (`out/gcc-asan/` and so on) and never touches the programs in `out/gcc/`.

2.5 verify.sh: the full check, and the pre-push hook

```
./verify.sh                 # gcc and clang
./verify.sh --compiler gcc  # one compiler
```

For each compiler it does a **debug build with every test**, then a **release build with every test**. It stops at the first failure and ends with **release builds in `out/`**, the programs the GUI runs. Each step's full output is in `out/verify/<compiler>-<type>.log`. It also checks

that each step really tested the type it claims. A run takes well under a minute when little has changed:

```
verify: debug, then release; every test; compilers: gcc clang
 gcc    debug    building and testing ... ok    10 s  passed: 296/296 checks
 clang  debug    building and testing ... ok     7 s  passed: 296/296 checks
 gcc    release  building and testing ... ok    13 s  passed: 303/303 checks
 clang  release  building and testing ... ok    13 s  passed: 303/303 checks
 took 43 s
 verify: all good; out/ holds release builds
```

The pre-push hook. `v001/setup/hooks/pre-push` runs `verify.sh` before every `git push`; a failure stops the push. `bootstrap.sh` sets it up; by hand:

```
git config core.hooksPath v001/setup/hooks
```

To push once without the check: `git push --no-verify`.

2.6 Install for root: TreeSyncInstall

You build as yourself; most syncs need root. Root must not run programs your user can change (anything running as you could swap them first), so the release programs are **installed** into the usual system folders, owned by root.

TreeSyncInstall (in the TreeSyncApp folder, or `v001/out/gcc/treesync-install`) does it. It runs as you, from the source tree. It has **one row per compiler**, GCC and Clang, each installed on its own, so you can install one or both:

- What the build in `out/` is: debug or release, and when it was built. Only a release build can be installed.
- Whether it is installed: *not installed*, *installed: this build*, or *installed: an older build* (then Install gives root the new one).
- **Install GCC...** / **Install Clang...**: runs `install.sh --compiler gcc` (or `clang`) as root (it asks for your password once per window: later Installs and Removes in the same window do not ask again), then puts the TreeSyncApp folder in your app list (`setup/app-folder.sh`). The other compiler's install stays as it is.
- **Remove GCC...** / **Remove Clang...**: removes that one; the other stays.

Below: **Installed now** (what `INSTALLED-gcc` and `INSTALLED-clang` say: build, commit, who, when) and the output of `install.sh`. **Show what it does** shows a dry run (no password). **Remove all...** removes everything it installed; your settings, databases and logs stay. **Help** (F1): this chapter. As root it refuses to start.

The same from a terminal:

```
./install.sh                              # both release builds; asks for your sudo
password
./install.sh --compiler gcc               # only the GCC one (or clang); the other stays
./install.sh --dry-run                    # only say what it would do
./install.sh --uninstall --compiler clang # remove only the Clang one
./install.sh --uninstall                  # remove it all again
```

Installed	Where
The programs	`/usr/local/bin/treesync-root-gcc`, `treesync-home-gcc`, `treesync-cli-gcc`, and the same with `-clang` (`root:root`, mode 755, like every program in `/usr/bin`)
`treesync-cli` for the terminal	`/usr/local/bin/treesync-cli`: a link to `treesync-cli-gcc`, or to `-clang` when only that one is installed
The manual	`/usr/local/share/doc/treesync/pdf/` (the Help menu finds it there)
What was installed	`/usr/local/share/doc/treesync/INSTALLED-gcc`, `INSTALLED-clang`: build, commit, who, when, the SHA-256 of its `treesync-cli`
App list entries	`/usr/local/share/applications/treesync-root-gcc.desktop`, `treesync-home-gcc.desktop` (and `-clang`): **TreeSyncRoot-GCC**, **TreeSyncHome-GCC**, **TreeSyncRoot-Clang**, **TreeSyncHome-Clang**, in the TreeSyncApp folder
polkit actions	`/usr/share/polkit-1/actions/org.treesync.run-as-root.policy`, one per compiler: TreeSyncRoot's runs ask for the password once, then not again for a few minutes

Each sync program runs the `treesync-cli` of its own compiler: TreeSyncRoot-Clang never runs the GCC engine. Both builds share your settings, database and logs.

It refuses a debug build: build a release in TreeSyncBuild (or `./verify.sh`, or `./build.sh --compiler gcc --release`) first. Programs go in under a temporary name and are renamed into place, so a TreeSync that is running keeps its copy. Older installs' programs and app entries without the compiler in their names (`treesync`, `treesync-root`, `treesync-home`) are removed.

Install again after every build you want to use as root. The installed copy does not follow `out/` by itself.

The installed copy only runs. TreeSyncBuild and TreeSyncInstall stay in the source tree. TreeSyncRoot syncs as root, for administrators; TreeSyncHome syncs as you, for anyone (see 3.9).

The TreeSyncApp folder. Every entry carries the category `X-TreeSyncApp`. `setup/app-folder.sh` (run by bootstrap and TreeSyncInstall, as you) writes the TreeSyncBuild and TreeSyncInstall entries in `~/.local/share/applications` and makes the folder: on GNOME an app folder in the app grid (`gsettings org.gnome.desktop.app-folders`), on other desktops a menu folder (`~/.config/menus/applications-merged/treesyncapp.menu`). `setup/app-folder.sh --remove` takes it away again. Each user who wants the folder runs it once; without it the entries are still in the app list, just not grouped.

2.7 Building as root

You do not need to, and TreeSyncBuild and TreeSyncInstall refuse to start as root. The scripts still work as root, through `sudo`, `pkexec` or a root login: they use the U++ of the user the TreeSync folder belongs to (root's home has none), and give everything they made in `out/` back to that user, so your own next build can overwrite it.

2.8 After changing code

1. Build and test while working: `./build.sh --compiler gcc --test`.
2. Before pushing (or let the hook do it): `./verify.sh`.
3. When threads, locks or memory handling changed, also the sanitizer runs (2.4).
4. To use the new build as root: TreeSyncInstall (or `./install.sh`).

`verify.sh` leaves release builds in `out/`, so the GUI is fast again afterwards.